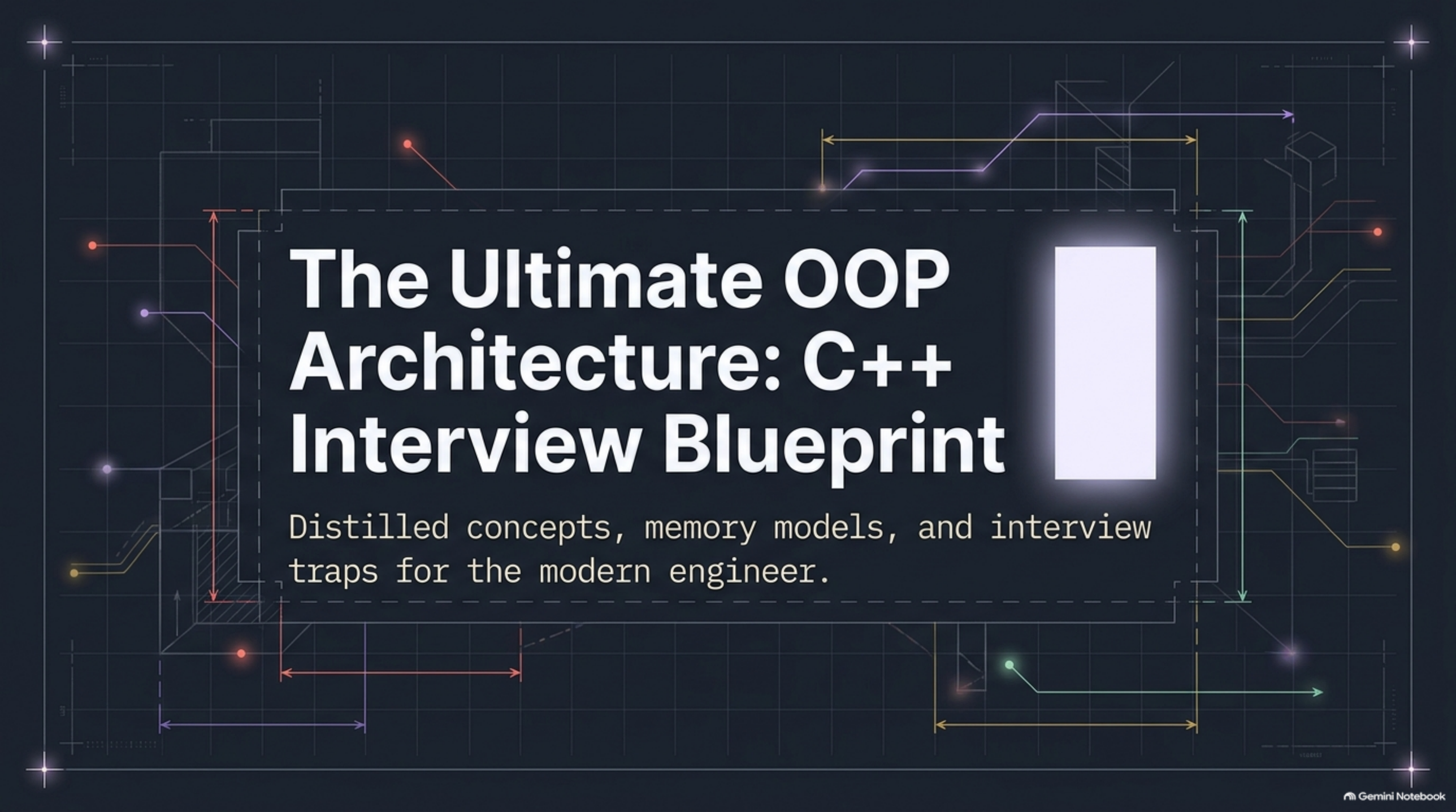
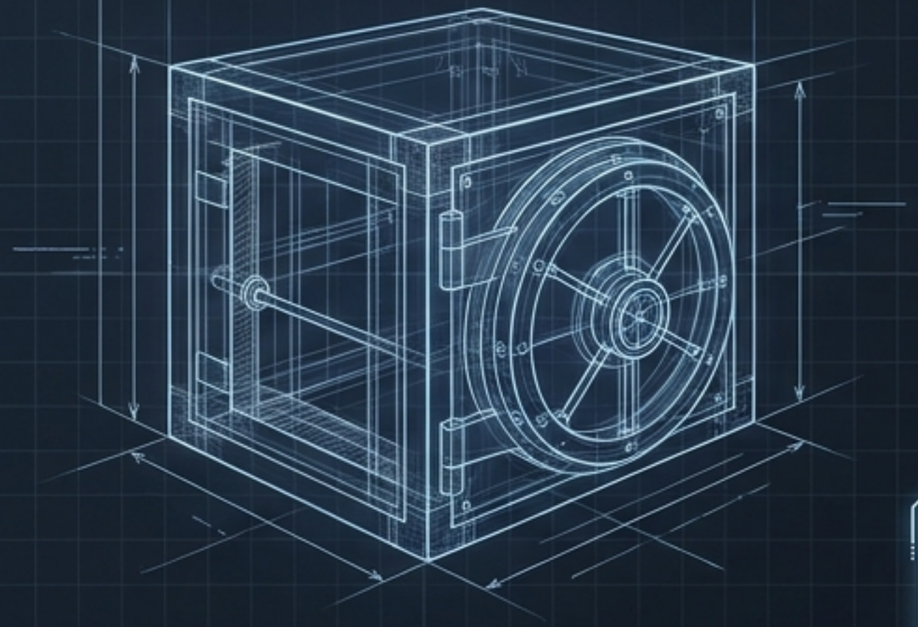


The Foundation: Classes, Objects, and State Management

The Blueprint

Class: BankAccount



- Blueprint/Template
- Defines State (Data)
- Defines Behavior (Methods)

The Physical Buildings

Object: account1

Object: account2

0x7FFF5FBFF8C0

0x7FFF5FBFF8E8

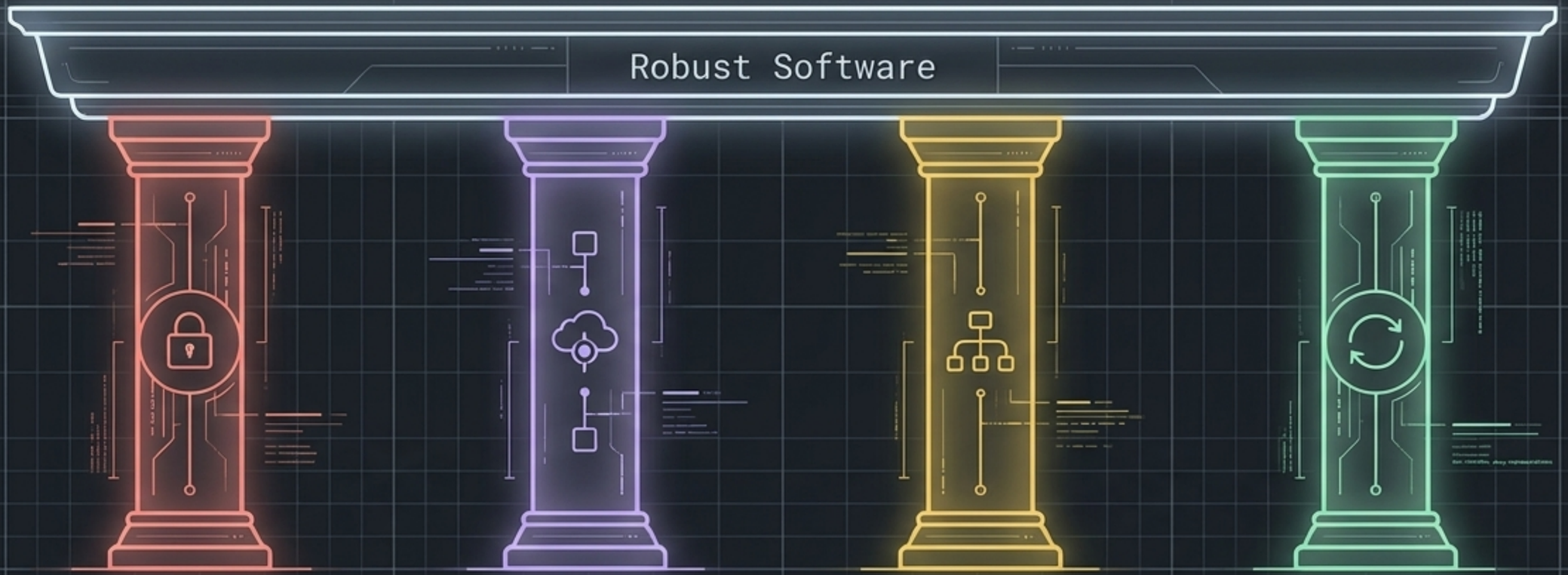
The this Pointer

```
void deposit(double amount) {
  // 'this' points to the current object
  this->balance += amount;
}
```

`this` is a constant pointer to the current object's memory.

Interview Trap: Static member functions belong to the Class (shared state) and do NOT have a **`this`** pointer!

The Four Pillars Form the Core Structure of OOP



Encapsulation

Data hiding & security

Abstraction

Focus on what, not how

Inheritance

Reuse & extend existing code

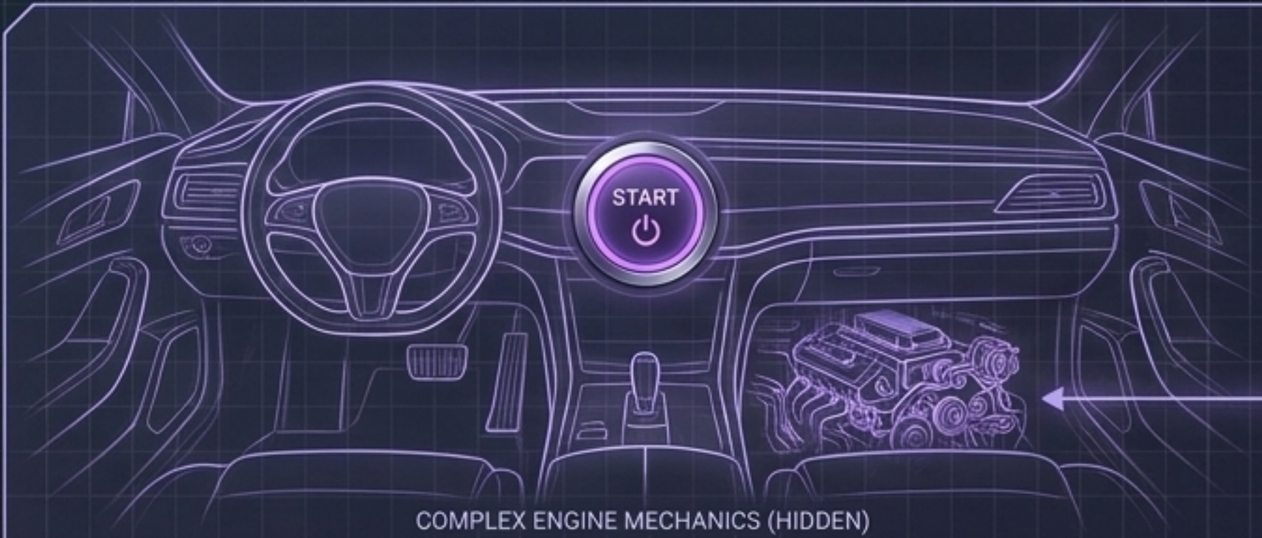
Polymorphism

One interface, many implementations

Encapsulation Protects State; Abstraction Hides Complexity

```
account.balance = -100000; // ERROR: Private
account.deposit(5000);     // OK: Public
```

`private`	Accessible only inside the class
`protected`	Accessible inside class + derived classes
`public`	Accessible from anywhere




```
virtual void start() = 0; // Pure Virtual Function
```

Exposes what an object does (the ignition button) while hiding the complex engine mechanics underneath.

Modeling Relationships: Inheritance vs. Composition

A dog is fundamentally a subset of animal. (Public inheritance).

Engine belongs to Car. When Car is destroyed, Engine dies.

Students can exist independently if Department closes.

The Golden Rule

Prefer Composition Over Inheritance when a true sub-type relationship doesn't exist. It lowers coupling.

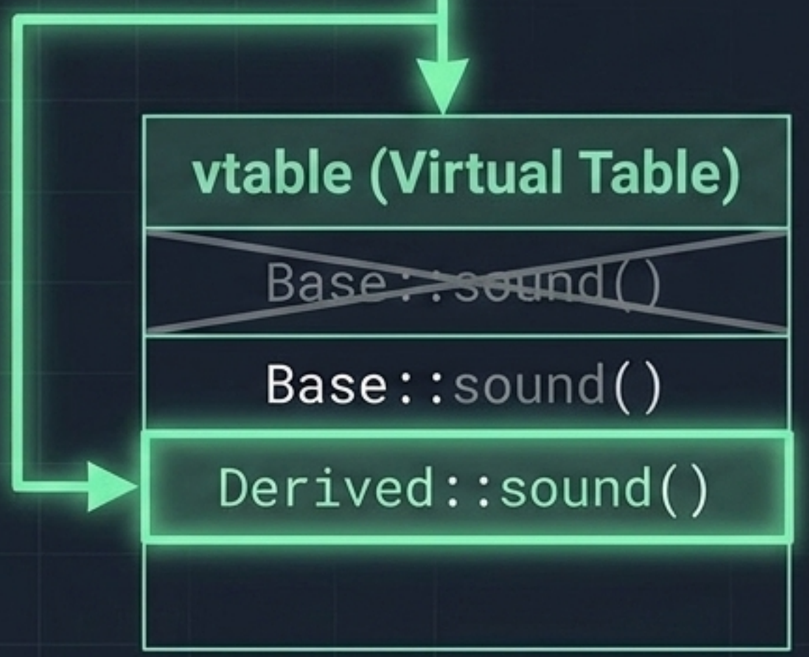
The Diamond Problem and the Virtual Inheritance Fix

Polymorphism: One Interface, Many Implementations

Compile-Time (Early Binding / Static)	Runtime (Late Binding / Dynamic)
Resolved by the compiler before the program runs. • **Function Overloading**: Same name, different parameters. • **Operator Overloading**: Changing '+' to add two `Point` objects. **Interview Trap:** You cannot overload `.`, `::`, `sizeof`, or `?:`. Prefix `++` uses `operator++()`; postfix uses dummy parameter `operator++(int)`.	Resolved dynamically while the program executes. • **Virtual Functions & Overriding**: Derived Derived class provides its own implementation of a base-class method.

Dynamic Dispatch Powers Runtime Polymorphism

```
Base* b = new Derived();
b->sound();
```



Virtual Function

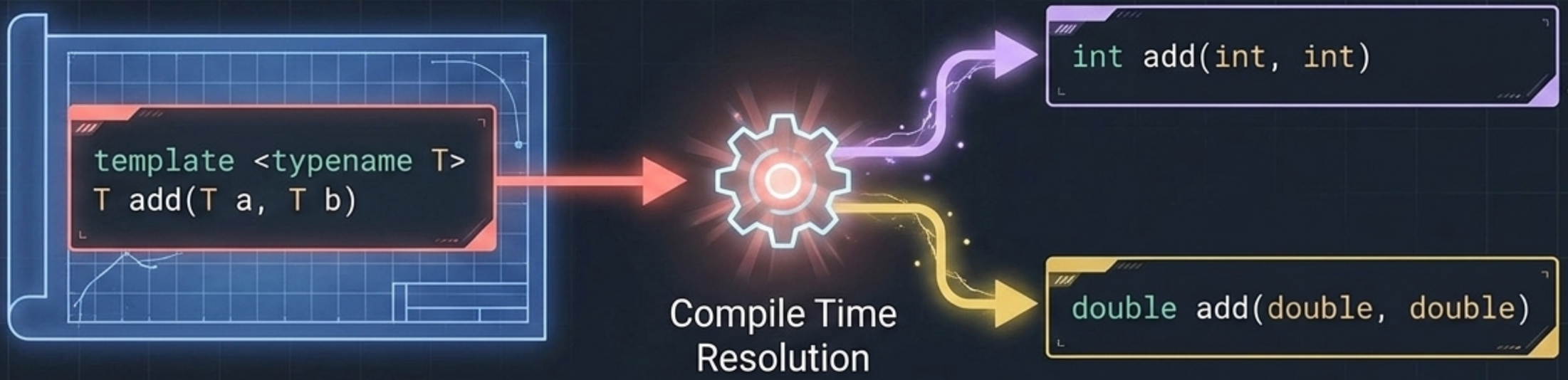
Has an implementation, but derived classes may override it for dynamic dispatch.

Pure Virtual Function (= 0)

Forces derived classes to implement it. Makes the parent an Abstract Class (cannot be instantiated directly).

This is how C++ models Interfaces.

Templates Execute Polymorphism at Compile Time



Templates (Static Polymorphism)	Virtual Functions (Dynamic Polymorphism)
Type known at compile time. Compiler physically generates new code for each type.	Implementation selected dynamically at runtime via vtable dispatch.

Object Lifecycles: Construction and Destruction

Called **automatically**. **Member Initializer Lists** (`Class() : x(1) {}`) are strictly required for const members and references.

Object exists on the **Stack** (automatic scope cleanup) or **Heap** (manual new/delete).

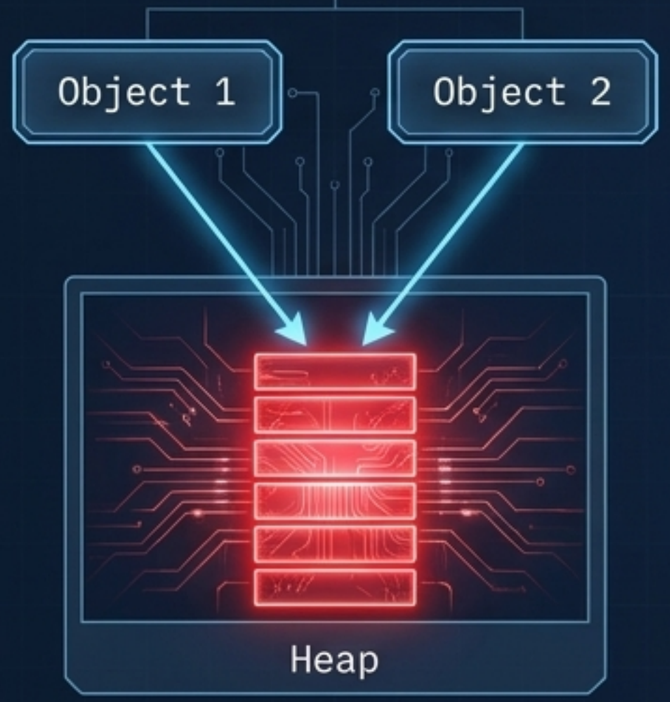
Cleans up resources and memory.

Interview Trap: Constructors vs. Virtual Destructors

- **Constructors cannot be virtual**. (The object doesn't fully exist yet to dispatch! Calling a virtual function here just runs the base version).
- **Polymorphic base classes must have virtual destructors**, otherwise deleting a derived object via a base pointer causes a massive resource leak.

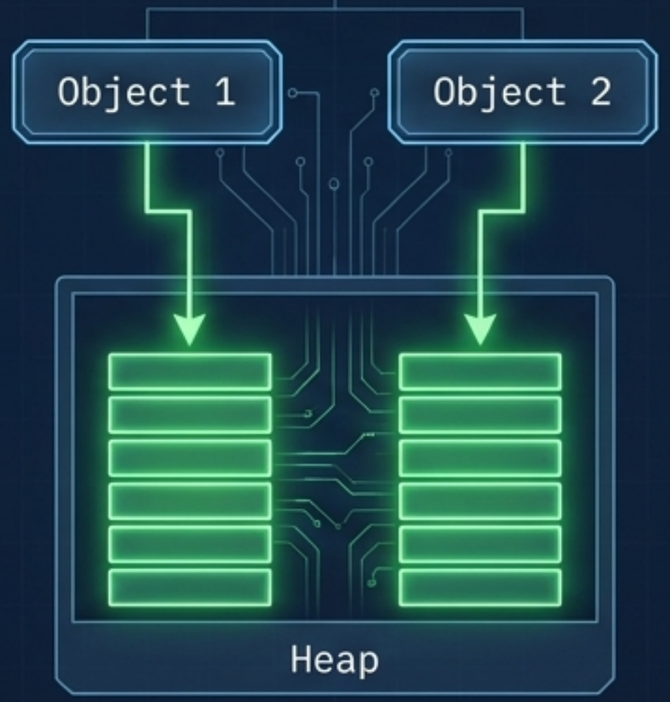
Memory Management Relies on Precise Copy and Move Semantics

Shallow Copy (Danger)



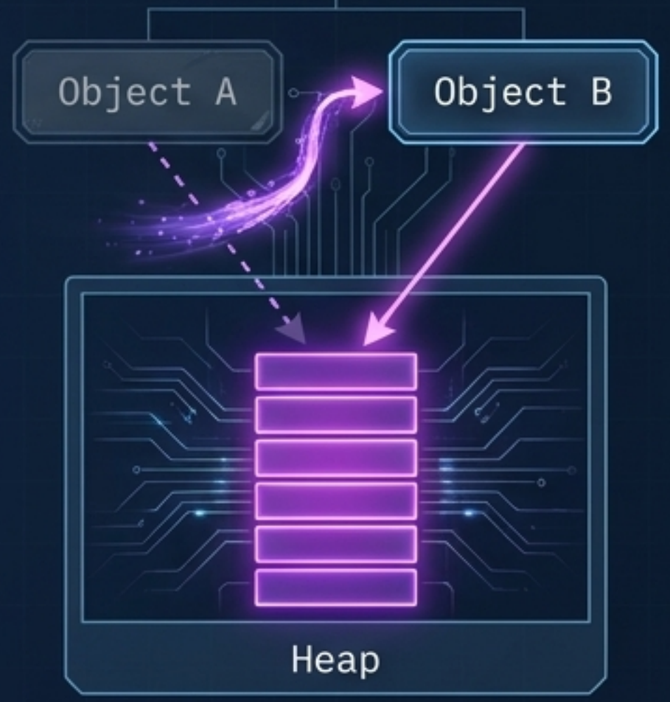
Both point to same memory.
Double-free crash imminent.

Deep Copy (Safe)



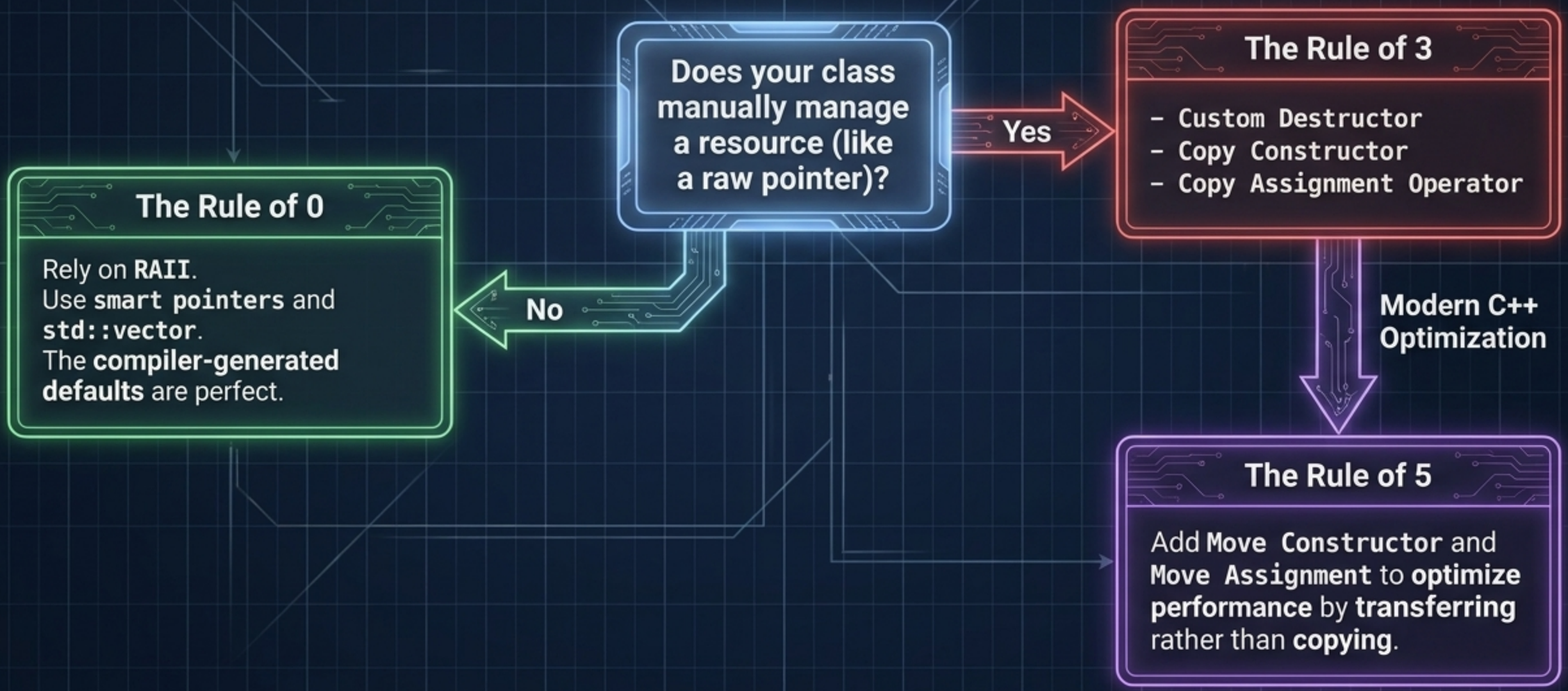
Independent memory blocks.
Safe destruction.

Move Semantics (`std::move`)

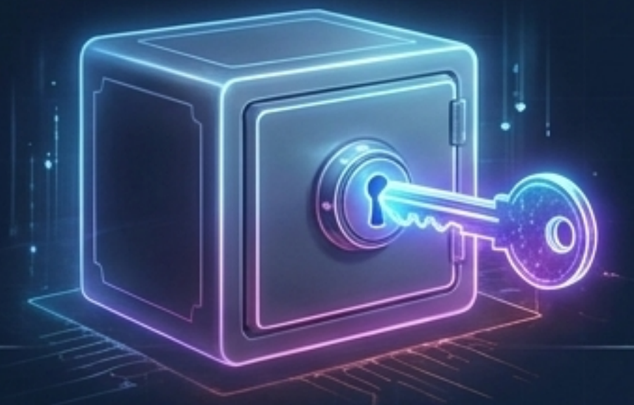


Transfers ownership of expensive resources without copying.

Modern C++ Resource Heuristics: The Rules of 3, 5, and 0



RAII and Smart Pointers Automate Resource Safety



1 owner. Cannot be copied, only moved.

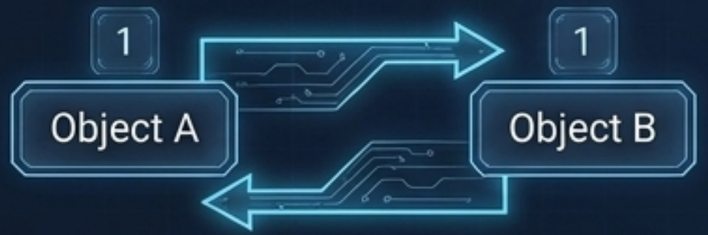
`unique_ptr<T>`

Shared ownership. Destroyed automatically when the reference count hits zero.

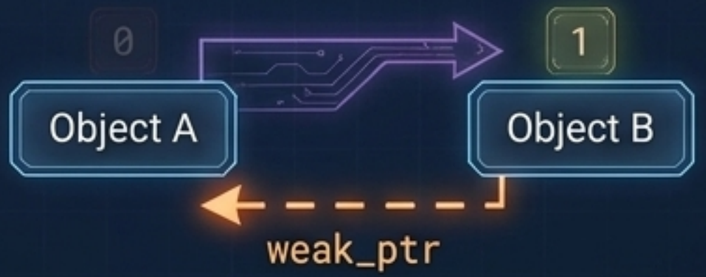
`shared_ptr<T>`

`shared_ptr<T>`

The Cyclic Reference Trap & weak_ptr Fix



`shared_ptr<B> a_ptr;` `shared_ptr<A> b_ptr;`



`weak_ptr` breaks the cycle. It observes the object without incrementing the reference count.

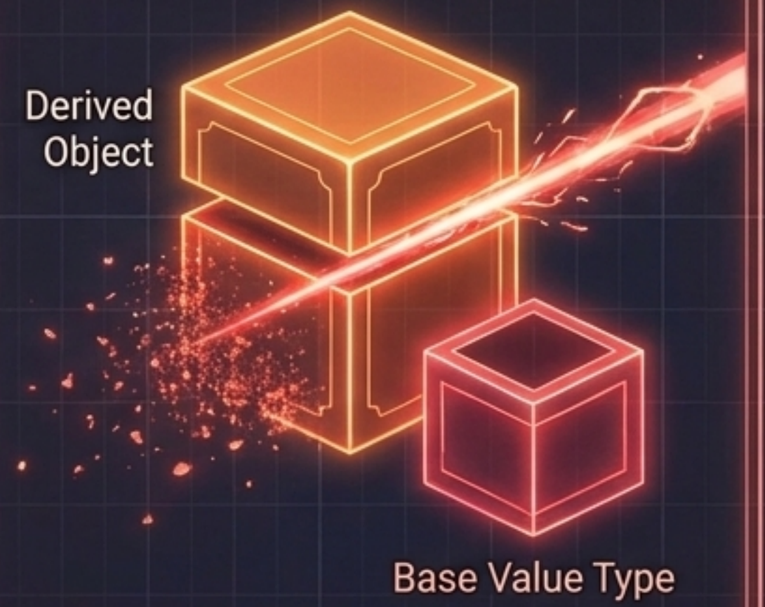
`weak_ptr<A> b_ptr;`

Const Correctness, Casting, and the Slicing Trap

Casting Matrix

`static_cast`	Compile-time, fast, no safety check for class hierarchies.
`dynamic_cast`	Runtime-checked safe downcasting, requires virtual functions.

Object Slicing Diagram



Data is lost. Avoid by passing via reference Base& or pointer Base*.

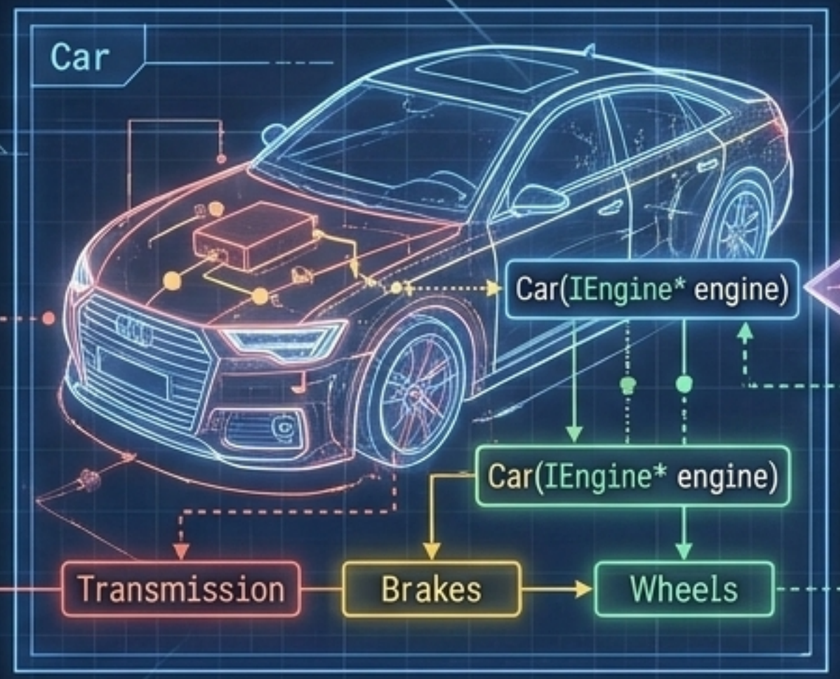
Const Correctness

```
int getAge() const;
```

const Student* const
(Read-only lock)

Note: use the 'mutable' keyword for exceptions like caching.

Master Synthesis: Orchestrating Architecture and Dependency Injection



High Cohesion: Classes should have one focused responsibility.

Low Coupling: Classes shouldn't heavily depend on each other's concrete implementations.

Dependency Injection: Inject interfaces (Polymorphism + Abstraction) via pointers rather than hardcoding components inside the object.

Mastering these structural interactions—not just the syntax—is the key to passing the C++ OOP architecture screen.